

SIMATS ENGINEERING

MODEL PRACTICAL EXAMINATION – Aug. 2026

COURSE NAME: Artificial Intelligence-Model Practical Examination

COURSE CODE: CSA1709

Register Number	192511053
Name	B Santhosh Kumar

Set 10

1. Write a python program to find goal state 6 using bidirectional search for the given graph.



2. Write a Python program to implement a two-layer feed-forward neural network to solve the XOR problem. The network should:
Use a single hidden layer with two neurons.
Use the sigmoid activation function.
Train the network using the backpropagation algorithm.

3. Write a Prolog program to predict weather conditions based on observed indicators.

Predicates:

hypothesis(Location, Condition): Predicts the weather condition for a location.

symptom(Location, Indicator): Observes weather indicators for a location.

Facts:

symptom(chennai, humid).

symptom(chennai, cloudy).

symptom(delhi, sunny).

symptom(delhi, dry).

symptom(ooty, cold).

symptom(ooty, foggy).

Rules:

hypothesis(chennai, rainy) :- symptom(chennai, humid), symptom(chennai, cloudy).

hypothesis(delhi, clear) :- symptom(delhi, sunny), symptom(delhi, dry).

hypothesis(ooty, foggy_weather) :- symptom(ooty, cold), symptom(ooty, foggy).

4. Write a Prolog program to predict a student's academic performance based on their behavior and attendance.

Predicates:

performance(Student, Level): Predicts the performance level of a student.

attribute(Student, Trait): Describes the traits of a student.

Facts:

attribute(john, hardworking).

attribute(john, regular).

attribute(sarah, irregular).

attribute(sarah, average).

attribute(mike, hardworking).

attribute(mike, irregular).

Rules:

performance(Student, excellent) :- attribute(Student, hardworking), attribute(Student, regular).

performance(Student, good) :- attribute(Student, hardworking), attribute(Student, irregular).

performance(Student, average) :- attribute(Student, average).

Laboratory Exam Component - 100 marks			
Programs (4)			
Description	Marks Distribution	Total Marks	Marks Obtained
Aim & Algorithm	2.5*4 = 10 Marks	10	
Programs	10*4 = 30 Marks	40	
Debugging	2.5*4 = 10 Marks	10	
Input & Output	05*4 = 20 Marks	20	
Result	2.5*4 = 10 Marks	10	
GitHub content	10 Marks	10	
Total Lab Exam Marks		100	

1. Bidirectional Search to Find Goal State 6

Aim

To write a Python program to find the goal state **6** from the starting state **0** using **Bidirectional Search**.

Algorithm

1. Represent the given graph using an adjacency list.
2. Start one search from the initial node 0.
3. Start another search from the goal node 6.
4. Expand nodes from both directions.
5. Check whether the two searches meet at a common node.
6. If they meet, combine both paths.
7. Display the path from the start node to the goal node.

Python Program

```
from collections import deque
```

```
graph = {  
    0: [4],  
    1: [4],  
    4: [0, 1, 6],  
    6: [4, 9],  
    9: [6, 11, 12],  
    11: [9],  
    12: [9]  
}
```

```
def bidirectional_search(start, goal):
```

```
    q1 = deque([start])
```

```
    q2 = deque([goal])
```

```
    parent1 = {start: None}
```

```
    parent2 = {goal: None}
```

```
    while q1 and q2:
```

```
node1 = q1.popleft()
```

```
for next_node in graph[node1]:
```

```
    if next_node not in parent1:
```

```
        parent1[next_node] = node1
```

```
        q1.append(next_node)
```

```
    if next_node in parent2:
```

```
        meet = next_node
```

```
        return make_path(meet, parent1, parent2)
```

```
# Search from goal
```

```
node2 = q2.popleft()
```

```
for next_node in graph[node2]:
```

```
    if next_node not in parent2:
```

```
        parent2[next_node] = node2
```

```
        q2.append(next_node)
```

```
    if next_node in parent1:
```

```
        meet = next_node
```

```
        return make_path(meet, parent1, parent2)
```

```
return None
```

```
def make_path(meet, parent1, parent2):
```

```
    path1 = []
```

```
    node = meet
```

```
    while node is not None:
```

```
        path1.append(node)
```

```
        node = parent1[node]
```

```
    path1.reverse()
```

```

node = parent2[meet]
while node is not None:
    path1.append(node)
    node = parent2[node]

return path1

start = 0
goal = 6
path = bidirectional_search(start, goal)
print("Start State:", start)
print("Goal State:", goal)

if path:
    print("Path:", " -> ".join(map(str, path)))
else:
    print("Goal not found")

```

Sample Output

```

Start State: 0
Goal State: 6
Path: 0 -> 4 -> 6

```

Debugging

- Checked the graph connections correctly.
- Used separate parent dictionaries for forward and backward searches.
- Verified that the two searches meet at a common node.
- Checked the final path from 0 to 6.

Result

Thus, the goal state **6** was successfully found from the starting state **0** using Bidirectional Search.

2. Two-Layer Feed-Forward Neural Network for XOR

Aim

To implement a **two-layer feed-forward neural network** with one hidden layer containing two neurons and train it using the **backpropagation algorithm** to solve the XOR problem.

Algorithm

1. Take the four XOR input combinations.
2. Initialize weights and biases randomly.
3. Use a hidden layer with two neurons.
4. Apply the sigmoid activation function.
5. Calculate the output of the network.
6. Find the error between actual and predicted output.
7. Apply backpropagation to update weights.
8. Repeat the process for several epochs.
9. Display the final XOR predictions.

Python Program

```
import numpy as np
```

```
# XOR input and output
```

```
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])
```

```
Y = np.array([
    [0],
    [1],
    [1],
    [0]
])
```

```
np.random.seed(1)
```

```

# Weights
W1 = np.random.randn(2, 2)
b1 = np.zeros((1, 2))

W2 = np.random.randn(2, 1)
b2 = np.zeros((1, 1))

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

learning_rate = 1
epochs = 10000

for i in range(epochs):

    # Forward propagation
    hidden = sigmoid(np.dot(X, W1) + b1)
    output = sigmoid(np.dot(hidden, W2) + b2)

    # Error
    error = Y - output

    # Backpropagation
    output_delta = error * sigmoid_derivative(output)

    hidden_error = np.dot(output_delta, W2.T)
    hidden_delta = hidden_error * sigmoid_derivative(hidden)

    # Update weights
    W2 += np.dot(hidden.T, output_delta) * learning_rate
    b2 += np.sum(output_delta, axis=0, keepdims=True) * learning_rate

```

```
W1 += np.dot(X.T, hidden_delta) * learning_rate
b1 += np.sum(hidden_delta, axis=0, keepdims=True) * learning_rate
```

```
print("XOR Output:")
for i in range(len(X)):
    print(X[i], "->", round(float(output[i][0])))
```

Sample Output

```
XOR Output: - -
[0 0] -> 0
[0 1] -> 1
[1 0] -> 1
[1 1] -> 0
```

Debugging

- Checked the input and expected XOR output.
- Verified that the hidden layer has exactly two neurons.
- Checked sigmoid calculation and its derivative.
- Verified that weights are updated during backpropagation.
- Increased the number of epochs to obtain correct XOR output.

Result

Thus, the two-layer feed-forward neural network was successfully trained using backpropagation and correctly solved the **XOR problem**.

3. Prolog Program to Predict Weather Conditions

Aim

To write a Prolog program to predict weather conditions based on observed weather indicators.

Algorithm

1. Define the symptom/2 predicate for weather indicators.
2. Store the given weather observations as facts.
3. Define rules using the observed symptoms.
4. Use the hypothesis/2 predicate to predict the weather condition.
5. Query the required location to obtain the prediction.

Prolog Program

symptom(chennai, humid).

symptom(chennai, cloudy).

symptom(delhi, sunny).

symptom(delhi, dry).

symptom(ooty, cold).

symptom(ooty, foggy).

hypothesis(chennai, rainy) :-

 symptom(chennai, humid),

 symptom(chennai, cloudy).

hypothesis(delhi, clear) :-

 symptom(delhi, sunny),

 symptom(delhi, dry).

hypothesis(ooty, foggy_weather) :-

 symptom(ooty, cold),

 symptom(ooty, foggy).

Sample Input

?- hypothesis(chennai, Weather).

hypothesis(delhi, Weather).

hypothesis(ooty, Weather).

Sample Output

```
| ?- consult('C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/Predict Weather.pl').
compiling C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/Predict Weather.pl for byte code...
C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/Predict Weather.pl compiled, 20 lines read - 1807 bytes written, 7 ms

(31 ms) yes
| ?- hypothesis(chennai, Weather).

Weather = rainy ?

(62 ms) yes
| ?- hypothesis(delhi, Weather).

Weather = clear ?

(31 ms) yes
| ?- hypothesis(ooty, Weather).

Weather = foggy_weather ?

(109 ms) yes
| ?- |
```

Debugging

- Checked that all facts use lowercase atoms.
- Checked the location names in the rules.
- Verified that all required symptoms are present before predicting the weather.
- Checked the predicate names and number of arguments.

Result

Thus, the Prolog program successfully predicted the weather condition based on the observed symptoms.

4. Prolog Program to Predict Student Academic Performance

Aim

To write a Prolog program to predict a student's academic performance based on their behavior and attendance attributes.

Algorithm

1. Define the attribute/2 predicate.
2. Store the student attributes as facts.
3. Define performance rules based on the student's attributes.
4. Use the performance/2 predicate for prediction.
5. Query the student's name to obtain the performance level.

Prolog Program

attribute(john, hardworking).

attribute(john, regular).

attribute(sarah, irregular).

attribute(sarah, average).

attribute(mike, hardworking).

attribute(mike, irregular).

performance(Student, excellent) :-

 attribute(Student, hardworking),

 attribute(Student, regular).

performance(Student, good) :-

 attribute(Student, hardworking),

 attribute(Student, irregular).

performance(Student, average) :-

 attribute(Student, average).

Sample Input

?- performance(john, Level).

?- performance(mike, Level).

Sample Output

```
| ?- consult('C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/Student academic.pl').  
compiling C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/Student academic.pl for byte code...  
C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/Student academic.pl compiled, 15 lines read - 1621 bytes written, 7 ms  
  
(15 ms) yes  
| ?- performance(john, Level).  
  
Level = excellence ? ;  
  
(15 ms) no  
| ?-  
performance(mike, Level).  
  
Level = good ?  
  
(16 ms) yes  
| ?-  
performance(sarah, Level).  
Level = good ;  
Level = excellence ;  
no
```

Debugging

- Checked that each student has the correct attributes.
- Checked the conditions in each performance rule.
- Verified the student name and performance predicate.
- Tested the program with John, Sarah and Mike.

Result

Thus, the Prolog program successfully predicted the academic performance of students based on their given attributes.